

51. N 皇后

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Hard	(Not Available)	(Not Available)	-	-	0

- **Tags:** 数组, 回溯
- **Companies:** (Not Available)

按照国际象棋的规则，皇后可以攻击与之处在同一行或同一列或同一斜线上的棋子。

n 皇后问题研究的是如何将 **n** 个皇后放置在 **n×n** 的棋盘上，并且使皇后彼此之间不能相互攻击。

给你一个整数 **n**，返回所有不同的 **n** 皇后问题的解决方案。

每一种解法包含一个不同的 **n** 皇后问题的棋子放置方案，该方案中 'Q' 和 '.' 分别代表了皇后和空位。

示例 1:

输入: **n** = 4

输出: `[[".Q..","...Q","Q...","..Q."],["..Q.","Q...","...Q",".Q.."]]`

解释: 如上图所示，4 皇后问题存在两个不同的解法。

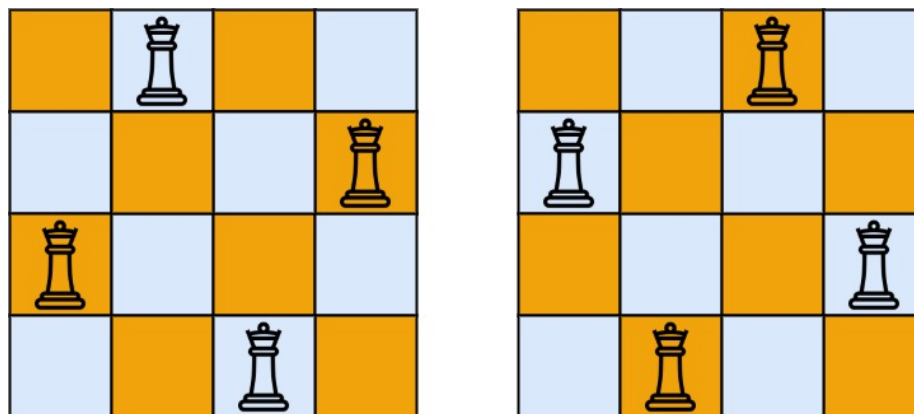


Figure 1: img

示例 2:

输入: **n** = 1

输出: `[["Q"]]`

提示:

- $1 \leq n \leq 9$

Discussion | Solution

解决方法

1. 问题理解

目标：在 $N \times N$ 的棋盘上放置 N 个皇后，使得任意两个皇后都不能互相攻击。找出所有不同的放置方案。攻击规则：皇后可以攻击同一行、同一列、同一条主对角线、同一条副对角线上的棋子。输出：一个列表，包含所有解决方案。每个解决方案是一个 $N \times N$ 的棋盘表示（列表的列表或字符串列表），‘Q’ 代表皇后，‘.’ 代表空位。

2. 思路分析

这是经典的 N 皇后问题，是回溯算法的绝佳应用场景。我们尝试逐行放置皇后。

决策树模型：- 树的每一层代表棋盘的一行。- 每一层的节点代表在该行尝试将皇后放置在哪一列。- 从根节点到叶子节点的路径（如果有效）构成一个完整的解决方案。

回溯过程：

1. 状态/路径 (State/Path):
 - 当前正在考虑在哪一行 `row` 放置皇后。
 - 记录已放置皇后的位置。一种有效的方式是用一个列表 `queens`，其中 `queens[r] = c` 表示第 `r` 行的皇后放在了第 `c` 列。
2. 选择列表 (Choices): 在当前行 `row`，可以尝试将皇后放置在哪一列 `col` (从 0 到 $N-1$)。
3. 约束条件/剪枝 (Constraints/Pruning): 在 (row, col) 放置皇后是否安全？即是否会被之前行 (0 到 `row-1`) 已放置的皇后攻击。
 - 列检查：当前列 `col` 是否已经被占用？
 - 主对角线检查：当前位置 (row, col) 所在的主对角线是否已被占用？对于主对角线上的点，`row - col` 的值是恒定的。
 - 副对角线检查：当前位置 (row, col) 所在的副对角线是否已被占用？对于副对角线上的点，`row + col` 的值是恒定的。
 - 行检查：由于我们是逐行放置，天然保证了同一行只有一个皇后，无需额外检查。
4. 高效约束检查：为了快速判断列和对角线是否被占用，可以使用集合 (Set):
 - `cols`: 存储已被占用的列索引。
 - `diag1`: 存储已被占用的主对角线标识 (`row - col`)。
 - `diag2`: 存储已被占用的副对角线标识 (`row + col`)。每次检查只需 $O(1)$ 时间。
5. 结束条件 (End Condition): 当成功放置了第 N 行的皇后 (即 `row == N`) 时，说明找到了一个完整的、有效的解决方案。

具体步骤:

1. 初始化:
 - 结果列表 `result = []`。
 - 用于记录占用的集合: `cols = set()`, `diag1 = set()`, `diag2 = set()`。
 - 用于记录皇后位置的列表: `queens = [-1] * n` (或类似结构)。
2. 定义回溯函数 `backtrack(row)`:
 - `row`: 当前要放置皇后的行号 (从 0 开始)。
3. 递归终止条件:
 - 如果 `row == n`:
 - 找到了一个完整解决方案。
 - 根据 `queens` 列表生成棋盘表示 `board`。
 - 将 `board` 加入 `result`。
 - 返回。
4. 遍历选择列表 (列):
 - 对于当前行 `row`, 遍历所有列 `col` (从 0 到 `n-1`)。
 - 检查约束:
 - 如果 `col` 在 `cols` 中, 或 `row - col` 在 `diag1` 中, 或 `row + col` 在 `diag2` 中, 则该位置不安全, `continue` 到下一列。
 - 如果位置安全:
 - 做选择:
 - * 记录皇后位置: `queens[row] = col`。
 - * 更新占用集合: `cols.add(col)`, `diag1.add(row - col)`, `diag2.add(row + col)`。
 - 递归进入下一层决策: 调用 `backtrack(row + 1)`。
 - 撤销选择 (回溯):
 - * 从占用集合中移除: `cols.remove(col)`, `diag1.remove(row - col)`, `diag2.remove(row + col)`。(注意: `queens[row]` 无需显式撤销, 因为下一轮循环或上一层递归会覆盖它)。
5. 生成棋盘表示函数 `generate_board(queens)`:
 - 输入 `queens` 列表。
 - 创建一个 `N x N` 的空棋盘 (用 `'.'` 填充)。
 - 根据 `queens[r] = c`, 将 `board[r][c]` 设置为 `'Q'`。
 - 将每一行转换为字符串。
 - 返回字符串列表表示的棋盘。
6. 主函数调用:
 - 调用 `backtrack(0)` 开始回溯。
 - 返回 `result`。

3. 代码实现 (Python)

```
from typing import List

class Solution:
    def solveNQueens(self, n: int) -> List[List[str]]:
```

```

"""
使用回溯算法解决  $N$  皇后问题。
Args:
    n: 棋盘大小和皇后数量。
Returns:
    所有不同的解决方案列表。
"""

result = [] # 存储所有解决方案
queens = [-1] * n # queens[row] = col 表示第 row 行的皇后放在 col 列

# 用于快速判断列和对角线是否被占用
cols = set() # 存储被占用的列
diag1 = set() # 存储被占用的主对角线 (row - col)
diag2 = set() # 存储被占用的副对角线 (row + col)

def generate_board() -> List[str]:
    """ 根据 queens 列表生成棋盘表示 """
    board = []
    for r in range(n):
        row_str = ['.'] * n
        if queens[r] != -1: # 确保该行已放置皇后
            row_str[queens[r]] = 'Q'
        board.append("".join(row_str))
    return board

def backtrack(row: int):
    """
    回溯函数核心逻辑。
    Args:
        row: 当前要放置皇后的行号。
    """
    # 递归终止条件: 成功放置了 n 行皇后
    if row == n:
        board = generate_board()
        result.append(board)
        return

    # 遍历当前行的所有列
    for col in range(n):
        # 检查约束条件: 列、主对角线、副对角线是否冲突
        if col in cols or (row - col) in diag1 or (row + col) in diag2:
            continue # 不安全, 尝试下一列

        # --- 位置安全, 做选择 ---
        queens[row] = col
        cols.add(col)

```

```

diag1.add(row - col)
diag2.add(row + col)

# --- 递归进入下一行 ---
backtrack(row + 1)

# --- 撤销选择 (回溯) ---
# queens[row] = -1 # 可选, 因为会被覆盖
cols.remove(col)
diag1.remove(row - col)
diag2.remove(row + col)

# 从第 0 行开始回溯
backtrack(0)
return result

# 示例调用
# sol = Solution()
# print(sol.solveNQueens(4))
# print(sol.solveNQueens(1))

```

4. 复杂度分析

- 时间复杂度: $O(N!)$
 - 这是一个粗略的上界。搜索空间的状态数大约是 $N!$ 级别。虽然不是严格的 $N!$, 因为剪枝会减少很多无效分支, 但最坏情况下的时间复杂度仍然与 $N!$ 相关。
 - 在每一层递归中, 我们需要遍历 N 列, 并进行 $O(1)$ 的约束检查 (使用集合)。
 - 生成每个解决方案需要 $O(N^2)$ 时间。
 - 总时间复杂度主要由搜索空间的大小决定, 约为 $O(N!)$ 。
- 空间复杂度: $O(N)$
 - 递归调用栈的深度最多为 N 。
 - `queens` 列表需要 $O(N)$ 空间。
 - `cols`, `diag1`, `diag2` 三个集合最多存储 N 个元素, 总共需要 $O(N)$ 空间。
 - 结果列表 `result` 的空间取决于解决方案的数量, 不计入算法本身空间复杂度。每个解决方案需要 $O(N^2)$ 空间。

5. 如何在面试中讲解

1. 问题理解:
 - “ N 皇后问题要求在 $N \times N$ 棋盘上放 N 个皇后, 使得它们互不攻击 (不同行、不同列、不同对角线)。目标是找到所有解。”
2. 核心思路: 回溯:
 - “这是一个典型的约束满足问题, 适合用回溯法。我们可以尝试按行放置

- 皇后。”
- “假设我们正在决定第 `row` 行皇后的位置。”
3. 状态与选择:
- “状态可以用当前行号 `row` 和已放置皇后的列信息（例如一个列表 `queens[r]=c`）来表示。”
 - “在第 `row` 行，我们的选择是尝试将皇后放在 0 到 `N-1` 的每一列 `col`。”
4. 约束检查 (关键):
- “在尝试放置 (`row`, `col`) 之前，必须检查该位置是否安全。”
 - “安全意味着：该列 `col` 没有被之前的皇后占用；该位置的主对角线 (`row - col`) 没有被占用；该位置的副对角线 (`row + col`) 没有被占用。”
 - “为了高效检查，可以使用三个集合 `cols`, `diag1`, `diag2` 分别记录已占用的列、主对角线标识、副对角线标识。这样检查只需 $O(1)$ 时间。”
5. 回溯过程:
- “定义 `backtrack(row)` 函数。”
 - “**Base Case:** 如果 `row == N`，表示 `N` 个皇后都已成功放置。生成当前棋盘布局，加入结果列表，返回。”
 - “循环与递归：遍历当前行 `row` 的所有列 `col`。”
 - “如果 (`row`, `col`) 安全（通过检查三个集合）：
 - * “做选择：将皇后放在 (`row`, `col`)，并更新三个集合。”
 - * “递归：调用 `backtrack(row + 1)` 处理下一行。”
 - * “撤销选择：递归返回后，将 (`row`, `col`) 的信息从三个集合中移除，恢复状态，以便尝试当前行的其他列。”
6. 结果生成:
- “当找到一个解时 (`row == N`)，需要根据记录的皇后位置（如 `queens` 列表）生成 `N x N` 的棋盘字符串列表。”
7. 初始调用:
- “从 `backtrack(0)` 开始。”
8. 复杂度分析:
- “时间复杂度大致是 $O(N!)$ ，因为搜索空间与 `N` 的阶乘相关。”
 - “空间复杂度是 $O(N)$ ，主要来自递归栈深度和辅助集合。”